Guía 2 · Explicación de todo el trabajo

Tablero de Indicadores · Panadería San Nicolás (pan trenza) — Proyecto de Aula 2026-2, Gerencia de Proyectos · estado al 27 de octubre de 2026

En una frase	La app se construye **desde cero en el repositorio**, una historia de usuario por commit, del 27 al 29 de octubre: prototipo v0 (OEE del horno) el martes 27 y prototipo final el jueves 29, ambos publicados en Google Apps Script.
Qué está listo	El código de las 16 historias, en paquetes listos para subir; 33 pruebas automáticas; documentación técnica en `Docs/`; el Word de historias de usuario; datos simulados de septiembre y octubre.
Qué falta	Que cada integrante suba su paquete en su fecha (Guía 1), publicar en Apps Script (Guía 3), la QA de María Camila y ensayar la demostración.

1. Qué cambió y por qué

Las historias anteriores estaban escritas como si la app ya existiera («ver el OEE», «filtrar»...), y el código se iba a subir de una sola vez. Eso no cumple lo que pide la guía: el historial de commits es la evidencia de los sprints. Ahora:

- Las **historias describen la construcción**: HU-01 crea la estructura vacía; HU-04 enseña a la app a leer el Excel; HU-06 calcula el OEE; ...; HU-16 publica la versión final.
- Cada historia es **un commit** con los archivos que realmente agrega, hecho por su responsable en la fecha real. Nada de cambiar fechas.
- La app vive en la estructura que el equipo ya creó en main: `Frontend/`, `Backend/` (Google Apps Script) y `Docs/`, más `Simulacion/` y `Tests/`.
- **Redacción mixta:** las historias de la base técnica se escriben desde el equipo («Como equipo de desarrollo, necesitamos...») y las de lo que ve la panadería desde el administrador («Como administrador de la panadería, quiero...»).

2. Cómo crece la app

Historia	Qué se ve en la app al terminarla
HU-01 Estructura y arquitectura	Página con Tablero y Cargar datos, vacía. README corregido (sin Google Sheets).
HU-02 Plantilla y diccionario	Botón «Descargar plantilla de Excel».
HU-03 Datos simulados	Botón «Descargar datos simulados» (septiembre de 2026).
HU-04 Leer el Excel	Zona para arrastrar el archivo; muestra las hojas y sus filas. Primeras pruebas automáticas.

HU-05 Reconocer y validar	Tablas reconocidas, conteos y filas con errores; «Quitar datos cargados».
HU-06 OEE del horno	**Prototipo v0:** el tablero aparece con OEE, disponibilidad, rendimiento y calidad. Publicado en Apps Script.
HU-07 · HU-08 · HU-09	Tarjetas de SLA, % no vendido y productividad.
HU-10 Semáforos y metas	Colores de las fichas técnicas y botón «Metas».
HU-11 Filtros	Desde, Hasta, Turno (y Equipo si hay varias máquinas).
HU-12 Gráficos	OEE por día, cascada, Pareto, cliente por día, productividad y OEE por turno.
HU-13 Hallazgos	Frases automáticas con lo más importante.
HU-14 Exportar e imprimir	Botones «Exportar CSV» e «Imprimir / PDF».
HU-15 Archivo distinto	Nada nuevo en pantalla: prueba con octubre y plan de pruebas.
HU-16 Prototipo final	**Prototipo final** en Apps Script y GitHub Pages; manual de usuario.

Cada etapa se probó en el navegador sin errores, y desde la HU-04 las pruebas automáticas pasan en cada commit: 3, 9, 14, 17, 20, 22, 25, 28, 31 y 33.

3. Arquitectura

```
Libro de Excel → Navegador: SheetJS + lectura.js → modelo.js → indicadores.js → app.js
+ Chart.js (tablero)
Google Apps Script (Backend/Code.gs) y GitHub Pages solo entregan la página.
```

Archivo	Qué hace	HU
`Frontend/index.html`, `css/`	La página: Tablero y Cargar datos	01
`Frontend/js/app.js`	La pantalla: carga, tarjetas, metas, filtros, gráficos, hallazgos, exportar	01 a 14
`Frontend/js/lectura.js`	Convierte textos, números con coma decimal y fechas de Excel	04
`Frontend/js/modelo.js`	Reconoce cada tabla por sus columnas, valida las filas y une Dim con Fact	05
`Frontend/js/indicadores.js`	Tiempos por turno, OEE, SLA, % no vendido, productividad, filtros, Pareto, semáforo	06 a 12
`Frontend/vendor/`	SheetJS (leer Excel) y Chart.js (gráficos): libres, incluidas para funcionar sin internet	04, 12
`Backend/`	`Code.gs` (doGet), `appsscript.json`, `construir_index.py` y el `Index.html` generado	01, 06, 16
`Simulacion/`	Montecarlo en Python y archivo de octubre	03, 15

`Tests/`	Un archivo de pruebas por historia	04 a 15

Por qué esta tecnología (lo que pidió el docente): costo $0, todo libre (MIT y Apache 2.0), el mismo JavaScript que el equipo ya usaba en Apps Script, sin depender de Google Sheets ni de la cuenta de un integrante, y el repositorio contiene todo. Apps Script se mantiene **solo para servir la página**: el código no vive allá, se genera desde el repositorio. Matriz ponderada en `Docs/Decisiones_Tecnicas.md`: 4,65 frente a 2,65 de Apps Script + Sheets.

4. Datos, indicadores y semáforos

El Excel tiene tablas de dimensiones (producto PR_01: 22 min por tanda de 16 trenzas; turnos T1 y T2 de 480 min con 30 de descanso; motivos de paro MP_01 a MP_08; operarios) y de hechos (producción por tanda, paros, pedidos, ventas del día y mano de obra). Detalle en `Docs/Diccionario_Datos.md`.

Indicador	Fórmula	Verde	Rojo
Disponibilidad	operativo ÷ programado; programado = 480 − 30 − paros planeados (MP_01, 02, 03, 08)	≥ 90 %	< 80 %
Rendimiento	Σ(unidades × 1,375 min) ÷ tiempo operativo	≥ 80 %	< 70 %
Calidad	buenas ÷ producidas	≥ 95 %	< 90 %
OEE del horno	A × R × Q	≥ 65 %	< 55 %
SLA (OTIF)	pedidos a tiempo y completos ÷ pedidos	≥ 95 %	< 90 %
% no vendido	(buenas − vendidas) ÷ buenas, por día	≤ 10 %	> 15 %
Productividad	buenas ÷ horas-hombre	≥ línea base + 5 %	< línea base

Los consolidados suman tiempos y unidades antes de dividir; no se promedian porcentajes.

5. Datos simulados (autorizados por el docente)

`Simulacion/simular_datos.py` aplica `Docs/Supuestos_Simulacion.md`: Montecarlo de los días hábiles × 2 turnos con Poisson, binomial, normal, lognormal, triangular y uniforme, con semilla fija. Cada libro trae una hoja *Verificacion* calculada aparte, y las pruebas comprueban que la app da exactamente lo mismo.

Indicador	Septiembre (desarrollo)	Octubre (archivo distinto)
OEE (A · R · Q)	67,7 % (89,9 · 78,6 · 95,8)	71,1 % (93,7 · 78,8 · 96,3)
SLA	92,1 %	87,7 %
% no vendido	2,3 %	3,7 %

Productividad	8,03 trenzas/h-h	8,42 trenzas/h-h

El % no vendido sale menor que el 8–10 % esperado porque el colchón del 10 % se aplica solo a la demanda de mostrador; es una inconsistencia de los supuestos, no de la app.

6. Respuesta a las observaciones del docente

Observación	Cómo se corrige
El código no está en el repositorio	Las 16 historias suben el código a `Frontend/` y `Backend/`, con pruebas en `Tests/`
No hay commits en las fechas de los sprints	Se reconoce sin falsificar fechas: desde el 27 de octubre cada historia es un commit y un Pull Request del responsable, en la fecha real
HU de documentación y varias numeraciones	Las historias del Word describen la construcción de la app (ver sección 8 sobre la numeración)
Trello no refleja la QA	Una tarjeta solo pasa a «Terminado» cuando QA prueba la historia en main
Sustentación débil	Arquitectura en `Docs/Arquitectura.md`; demostración con la app real y el archivo del docente
Justificar Apps Script / Sheets	Matriz ponderada por costo, licencias, curva, dependencia y sostenibilidad; Sheets se elimina y Apps Script solo sirve la página

7. Archivos entregados

Archivo	Para qué
`Paquetes_por_historia.zip`	Las 16 carpetas para subir en orden (Guía 1)
`Historias_de_Usuario_Construccion_App.docx`	Anexo del informe: las 16 historias con criterios, fecha y evidencia en el repositorio
Guías 1, 2 y 3 (PDF)	Subir a GitHub, entender el trabajo y publicar en Apps Script
Excel de septiembre, octubre y plantilla	Probar la app y ensayar la demostración

8. Decisión pendiente: la numeración

> En main, `Docs/Sprint_Planning.md` usa **HU-01 a HU-26** para tareas del informe (SIPOC, PESTEL, financiación…), y las historias de la app también son HU-01 a HU-16. Si quedan así, los números chocan, que es justo lo que observó el docente. Opciones: (a) renombrar las del informe como entregables **E-01 a E-26** en Sprint_Planning y en el Word de la segunda entrega (recomendado: son paquetes de la EDT, no historias de usuario); o (b) numerar las de la app como HU-27 a HU-42. Hay que decidirlo antes de subir la HU-01.

9. Preparación de la sustentación

- Mostrar la app real: la URL `/exec` de Apps Script (o GitHub Pages). Plan B: abrir `Frontend/index.html` sin internet.
- Arquitectura en 2 minutos: «Corre en el navegador; reconoce las tablas del Excel por sus columnas, valida y calcula los cuatro indicadores; Apps Script solo sirve la página; 33 pruebas comparan con cálculos independientes».
- Mostrar el historial de Git: un commit por historia, del prototipo v0 al final.
- Demo: Cargar datos → archivo del docente → Tablas reconocidas → Tablero → filtrar turno → Pareto y cascada → decisión de mejora.